

셰이더 프로그래밍

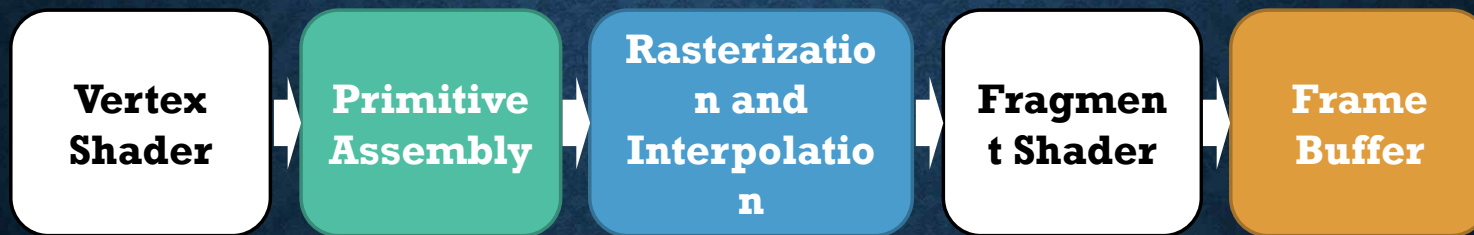
LECTURE7

2026년 1학기

담당교수 : 게임공학과 이택희

개요

- Framebuffer (프레임 버퍼)



FRAMEBUFFER

FRAMEBUFFER

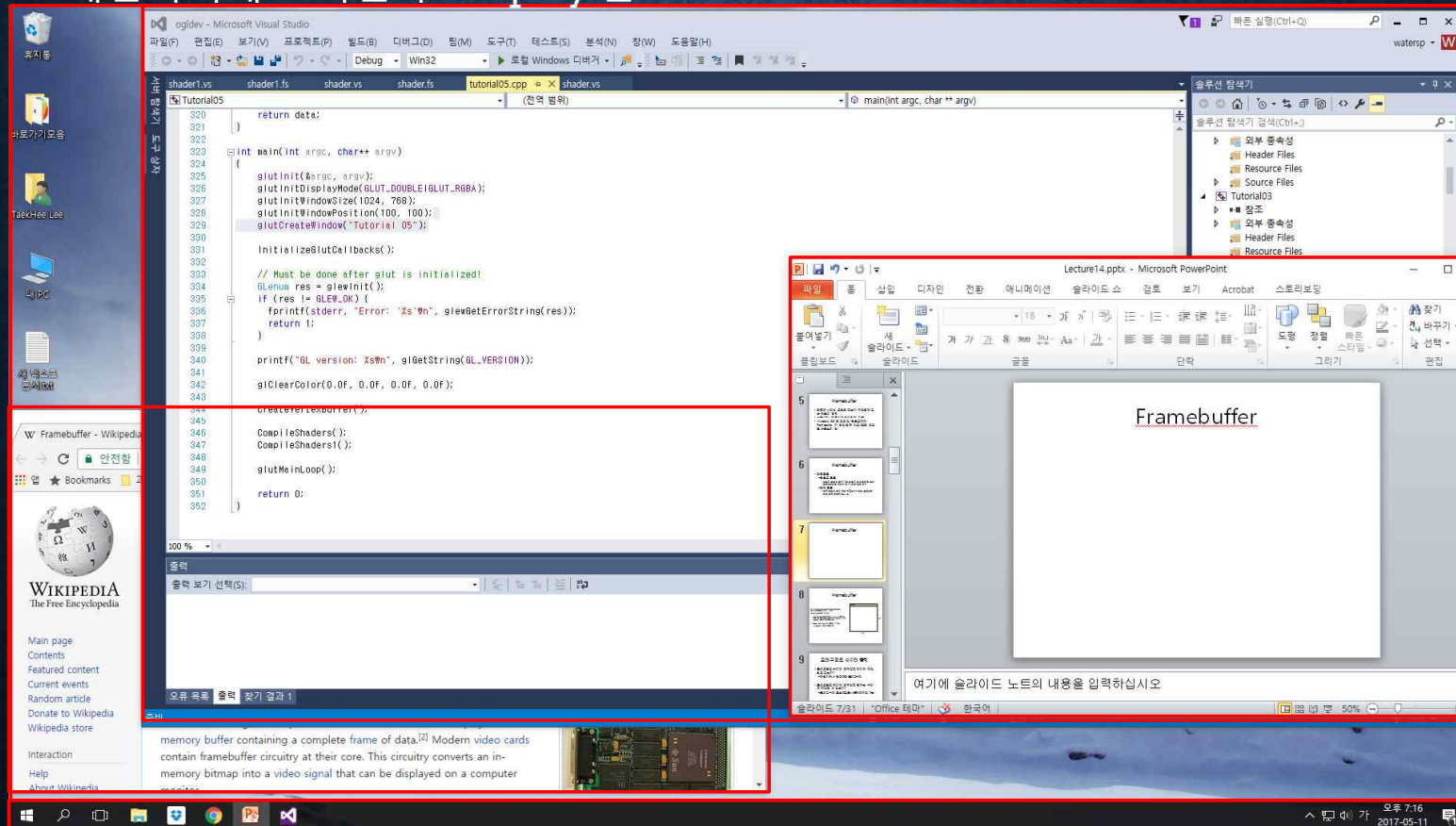
- 화면에 나타낼 그림의 정보가 저장되어 있는 메모리 영역
- 보통 GPU 메모리에 할당되어 있음
- Windows OS 의 경우 각 윈도우마다 Framebuffer 가 할당 되며 다른 OS의 경우도 대동소이 함

FRAMEBUFFER

- 화면 모드
 - 윈도우 모드
 - 윈도우 모드일 경우 다른 윈도우 및 바탕화면 등의 프레임버퍼와 합성이 된 후 최종 결과 출력
 - 전체 모드
 - 전체 모드일 경우 해당 어플리케이션의 프레임버퍼만 전체 화면에 표시 됨

FRAMEBUFFER

윈도우 화면에서의 프레임버퍼: 총 3개의 프레임버퍼들이 합성되어 최종 프레임버퍼에 그려진 후 Display 됨

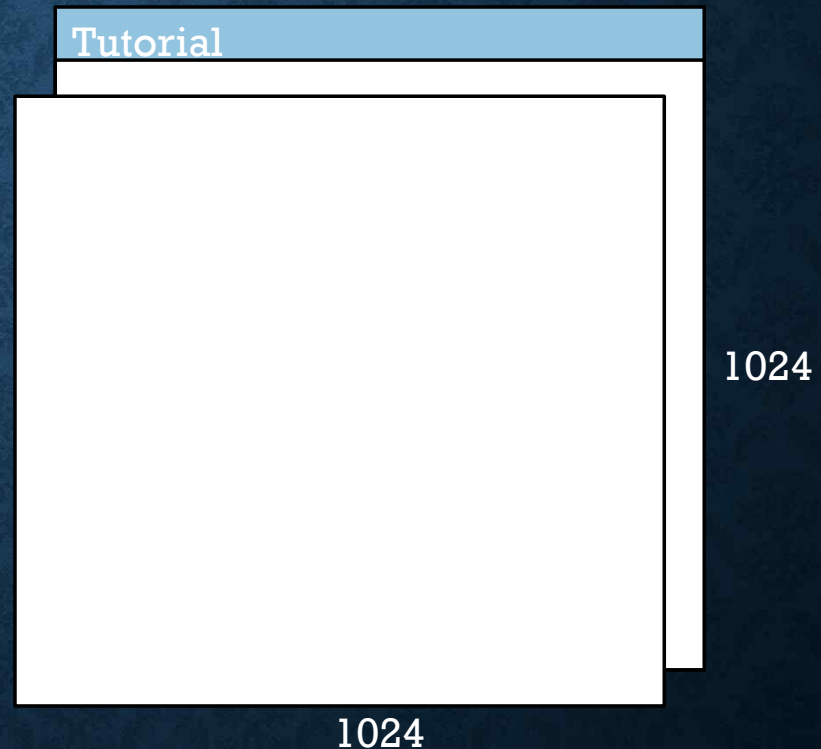


FRAMEBUFFER

- 윈도우가 가지고 있는 프레임버퍼에 OpenGL이 그림을 그릴 수 있게 하는 작업은 복잡함
- `glut` 에선 복잡한 과정을 아래 함수로 간략화 하여 사용할 수 있는 기능을 제공함

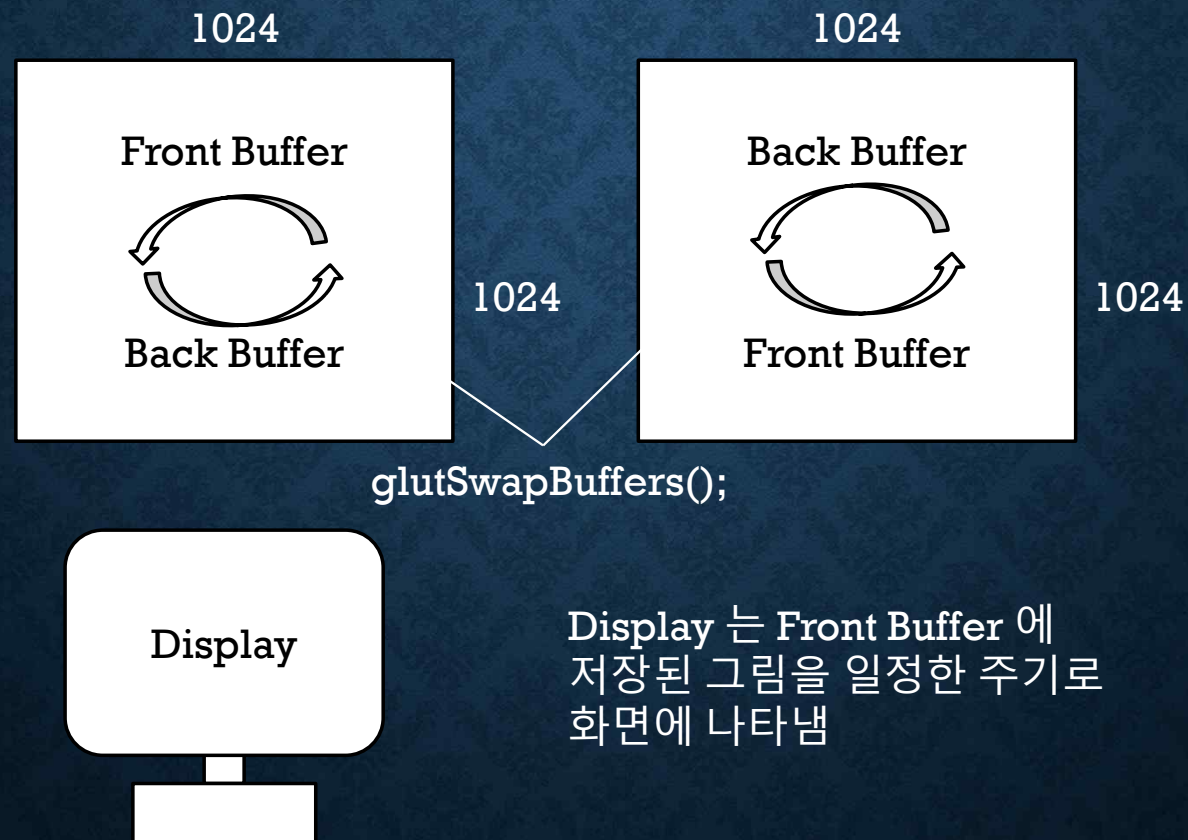
```
glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGBA);  
glutInitWindowSize(1024, 1024);  
glutCreateWindow("Tutorial");
```

- 채널 당 RGBA 값을 가지는 1024X1024 크기의 윈도우를 생성하고 윈도우의 `Framebuffer` 를 OpenGL 렌더링 타겟으로 설정
- Double buffering 이기 때문에 두 개의 `Framebuffer` 들이 만들어 짐



프레임버퍼 오브젝트

Double buffering 원리



FRAMEBUFFER

Fragment Shader

```
#version 330

in vec2 vTexPos;

layout(location=0) out vec4 FragColor;

void main()
{
    FragColor = texture(uTexture, vTexPos);
}
```

- 메인 프레임버퍼는 OpenGL 윈도우 생성시 자동으로 0번 location 에 bind 됨
- 따라서 지금까지 따로 설정을 하지 않아도 Default 값인 0번 location 으로 출력되고 있었음

FRAMEBUFFER

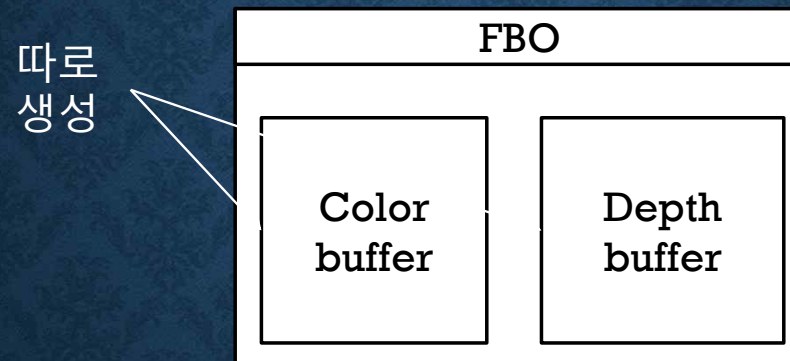
- 윈도우 시스템에서 제공하는 버퍼 이외의 메모리 영역에 그림을 그려야 한다면?
 - 메인 프레임 버퍼는 화면에 무조건 출력이 되기 때문에 그림자, 깊이 정보 등 최종 결과물이 아닌 결과가 저장되기에는 적합 치 않음
 - 따라서 윈도우가 관리하지 않는 프레임버퍼를 생성하여 그림을 그려놓을 수 있는 방법이 필요함 (Off-screen buffer)

FRAMEBUFFER

- 화면에 출력이 안 되는 버퍼를 위한 기능
 - Framebuffer Object(프레임버퍼 오브젝트)

프레임버퍼 오브젝트

프레임버퍼 오브젝트



FBO 자체는 껍데기에 불과하며 내용물을 따로 만들어서 attach 해 줘야 함
즉, FBO 생성 → Color buffer attach → Depth buffer attach 와
같이 따로 생성한 Buffer 들을 attach 해야 동작함

프레임버퍼 오브젝트

- FBO에 채울 내용물 만들기: Color buffer
 - Texture 를 Color buffer 로 attach 할 수 있음
 - Texture 생성 및 메모리 할당 필요
- `GLuint textureId; glGenTextures(1, &gFBOTexture);`
- `glBindTexture(GL_TEXTURE_2D, gFBOTexture);`
- `glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);`
- `glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);`
- `glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);`
- `glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);`
- `glTexParameteri(GL_TEXTURE_2D, GL_GENERATE_MIPMAP, GL_TRUE);`
- `glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA8, 512, 512, 0, GL_RGBA, GL_UNSIGNED_BYTE, 0);`

프레임버퍼 오브젝트

- FBO에 채울 내용물 만들기: Depth buffer
 - Depth buffer 의 경우 따로 Attach를 하지 않아도 기본 depth buffer 를 기반으로 동작함
 - 하지만 메인 프레임버퍼에 관련된 Depth 정보를 덮어버리기 때문에 따로 생성하여 사용하는 것이 안전함
- Render buffer 생성
 - `GLuint depthBuffer;`
 - `glGenRenderbuffers(1, &depthBuffer);`
 - `glBindRenderbuffer(GL_RENDERBUFFER, depthBuffer);`
 - `glRenderbufferStorage(GL_RENDERBUFFER, GL_DEPTH_COMPONENT, 512, 512);`
 - `glBindRenderbuffer(GL_RENDERBUFFER, 0);`

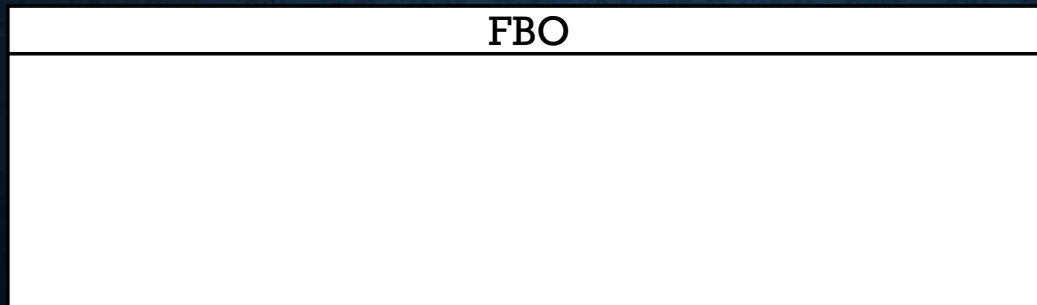
프레임버퍼 오브젝트

- FBO 생성

- `glGenFramebuffers(GLsizei n, GLuint* ids)`
- 텍스처나 VBO를 생성하는 방식과 유사함
- `glGenFramebuffers(1, &FBO0);`

- `glBindFramebuffer(GL_FRAMEBUFFER, m_A_FBO);` /attach전에 bind!

공백 FBO
생성됨



프레임버퍼 오브젝트

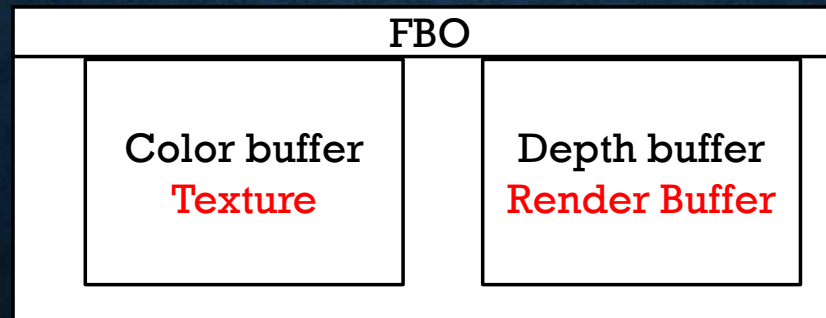
- FBO에 내용물 채우기

- Texture

- `glFramebufferTexture2D(GL_FRAMEBUFFER, GL_COLOR_ATTACHMENT0, GL_TEXTURE_2D, gFBOTexture, 0);`

- Depth buffer

- `glFramebufferRenderbuffer(GL_FRAMEBUFFER, GL_DEPTH_ATTACHMENT, GL_RENDERBUFFER, depthBuffer);`



프레임버퍼 오브젝트

에러 체크

```
GLenum status = glCheckFramebufferStatus(GL_FRAMEBUFFER);  
if(status != GL_FRAMEBUFFER_COMPLETE)
```

...

```
glBindFramebuffer(GL_FRAMEBUFFER, 0); /attach 후 원상복귀!
```

프레임버퍼 오브젝트

- `glViewport(GLint x, GLint y, GLsizei width, GLsizei height)`
 - 프레임버퍼 안에 그림을 그릴 영역을 설정해 주는 함수
 - 지금까지는 `default` 로 메인프레임 버퍼 크기만큼으로 설정되어 있었음

FBO 사용

FBO 사용

- 기존 메인 프레임 버퍼에 그리는 것이 아닌 따로 생성한 버퍼에 렌더링 하는 작업
 - 그려질 영역이 제대로 설정되어 있는지 확인 필요
 - 초기화가 잘 되도록 구성(ex: buffer clear)
- `glBindFramebuffer(GL_FRAMEBUFFER, FBOID);`
- `glViewport(0, 0, width, height);`
- 사용이 끝난 후에는 반드시 `main framebuffer` 로 초기화 해주는 것이 좋음
 - `glBindFramebuffer(GL_FRAMEBUFFER, 0);` //초기화 코드

프레임버퍼 오브젝트

1024



```
glViewport(0, 0, 1024, 1024);
```

1024

프레임버퍼 오브젝트

1024



`glViewport(0, 0, 512, 512);`

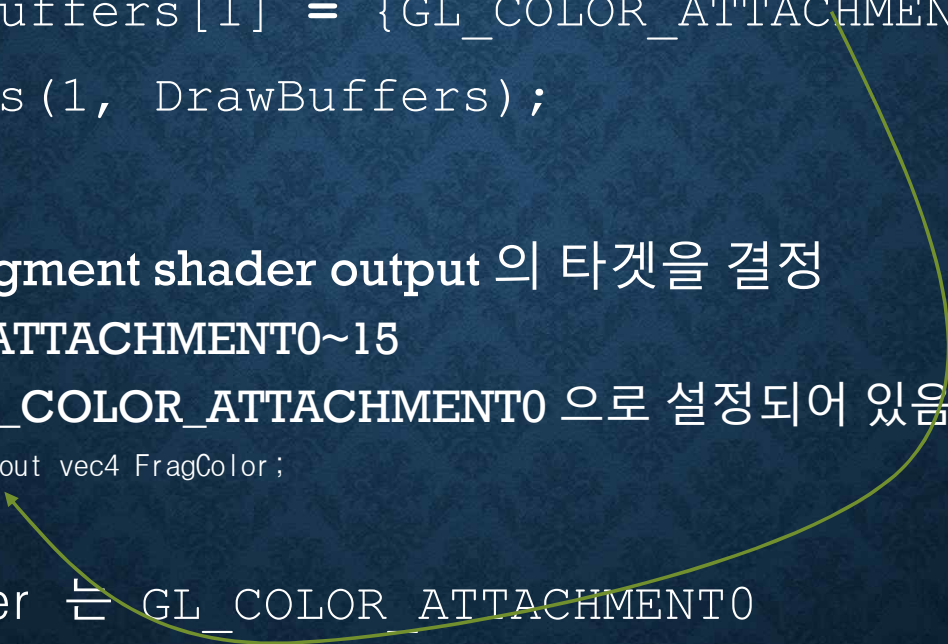
1024

MULTIPLE RENDER TARGETS

MULTIPLE RENDER TARGETS

- Fragment Shader에선 여러 개의 attach된 textures에 동시에 출력이 가능
 - Attachment0 : Color output
 - Attachment1 : Emissive output
 - Attachment2 : Depth output
- 왜 동시 출력을 하는가?
 - 렌더링 결과물 외에 중간과정에서 생기는 다른 결과물을 활용할 필요가 있을 때
 - 이미 만들어진 결과물의 경우 다시 렌더링하는 것 보단 이미 만들어진 결과를 저장하고 쓰는 것이 이득

MULTIPLE RENDER TARGETS

- `GLenum DrawBuffers[1] = {GL_COLOR_ATTACHMENT0};`
 - `glDrawBuffers(1, DrawBuffers);`
 - **DrawBuffer** : fragment shader output 의 타겟을 결정
 - `GL_COLOR_ATTACHMENT0~15`
 - Default 로 `GL_COLOR_ATTACHMENT0` 으로 설정되어 있음
 - `layout(location=0) out vec4 FragColor;`
 - Main framebuffer 는 `GL_COLOR_ATTACHMENT0`
- 

MULTIPLE RENDER TARGETS

- 두개의 outputs

- `GLenum DrawBuffers[2] = {GL_COLOR_ATTACHMENT0, GL_COLOR_ATTACHMENT1};`
- `glDrawBuffers(2, DrawBuffers);`

- 기존 코드를 활용하여 output 테스트

실습

실습

- 지난 시간 코드에 이어서 작성
 - 지금까지 진행했던 결과를 FBO 에 그린 후 FBO 에 attach 되어 있는 texture 를 사용하여 메인 프레임에 네 개의 결과물이 각각 그려지도록 구현